FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Julius Jojo C. Sabulao Jaimin Jud Roces
Section:	AN43
Professor:	Crisola Tan
Date:	11/3/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

A. Program File

```
import json
from pathlib import Path

CASE_FILE = Path("cases.json")

# Case representation: previous disaster cases
cases = [
  {
    "id": "C001",
    "problem": {
      "type": "flood",
      "severity": 4,
      "population": 12000,
      "rainfall": 260,
      "road": "limited"
    },
    "solution": {
      "site": "North High School",
      "teams": 4,
      "trucks": 2,
      "action": "Evacuate low-lying households first"
    }
  },
  {
    "id": "C002",
    "problem": {
      "type": "flood",
      "severity": 5,
      "population": 25000,
      "rainfall": 420,
      "road": "blocked"
    },
```

```
        "solution": {
            "site": "East Sports Complex",
            "teams": 8,
            "trucks": 4,
            "action": "Evacuate all zones immediately"
        }
    },
    {
        "id": "C003",
        "problem": {
            "type": "earthquake",
            "severity": 4,
            "population": 18000,
            "rainfall": 40,
            "road": "limited"
        },
        "solution": {
            "site": "Central University Gym",
            "teams": 7,
            "trucks": 3,
            "action": "Search damaged buildings and assist injured residents"
        }
    }
]

# New disaster problem
new_case = {
    "type": "flood",
    "severity": 5,
    "population": 22000,
    "rainfall": 390,
    "road": "blocked"
}


# Similarity assessment
def similarity(query, case):
    score = 0

    if query["type"] == case["type"]:
        score += 0.30

    score += max(
        0,
```

```
        1 - abs(query["severity"] - case["severity"]) / 4
    ) * 0.20

    score += max(
        0,
        1 - abs(query["population"] - case["population"]) / 30000
    ) * 0.20

    score += max(
        0,
        1 - abs(query["rainfall"] - case["rainfall"]) / 500
    ) * 0.20

    if query["road"] == case["road"]:
        score += 0.10

    return score


# Display the new problem
print("NEW DISASTER PROBLEM")
print("--------------------")
print("Disaster type:", new_case["type"])
print("Severity:", new_case["severity"])
print("Affected population:", new_case["population"])
print("Rainfall:", new_case["rainfall"], "mm")
print("Road access:", new_case["road"])

# Retrieve the most similar case
ranked_cases = sorted(
    cases,
    key=lambda case: similarity(new_case, case["problem"]),
    reverse=True
)

retrieved_case = ranked_cases[0]
score = similarity(new_case, retrieved_case["problem"])

# Adapt the retrieved solution
adapted_solution = retrieved_case["solution"].copy()

if new_case["population"] > 20000:
    adapted_solution["teams"] += 2
    adapted_solution["trucks"] += 1
```

```
if new_case["severity"] >= 5:
    adapted_solution["action"] = (
        "Begin full emergency evacuation immediately"
    )

if new_case["road"] == "blocked":
    adapted_solution["action"] += (
        " using boats, helicopters, or alternate routes"
    )

# Display the CBR result
print("\nCASE-BASED REASONING RESULT")
print("---------------------------")
print("Retrieved case:", retrieved_case["id"])
print("Similarity score:", round(score, 3))

print("\nADAPTED SOLUTION")
print("----------------")
print("Evacuation site:", adapted_solution["site"])
print("Rescue teams:", adapted_solution["teams"])
print("Water trucks:", adapted_solution["trucks"])
print("Recommended action:", adapted_solution["action"])

# Retain the new case
new_case_record = {
    "id": "C004",
    "problem": new_case,
    "solution": adapted_solution,
    "outcome": "Residents were evacuated safely."
}

cases.append(new_case_record)

with CASE_FILE.open("w", encoding="utf-8") as file:
    json.dump(cases, file, indent=4)

# Display learning result
print("\nLEARNING AND RETENTION")
print("----------------------")
print("New case retained successfully.")

print("\nUPDATED CASE BASE")
print("-----------------")
```

```
for case in cases:
    print(case["id"], "-", case["problem"]["type"])

print("Total cases in case base:", len(cases))
```

B. Short Documentation

Disaster Response Decision Support System Using Case-Based Reasoning

Description of the Case Base

The case base contains previous disaster-response situations involving floods and earthquakes. Each case is represented using a dictionary containing the disaster type, severity, affected population, rainfall amount, road accessibility, evacuation site, number of rescue teams, number of water trucks, and recommended emergency action. The system contains three previous cases. These cases represent different disaster conditions and their corresponding response plans. When a new disaster occurs, the system compares the new problem with the stored cases and retrieves the most similar one.

Explanation of the Similarity Method

The system uses a weighted similarity method. The features and their weights are:

- Disaster type: 30%
- Severity: 20%
- Affected population: 20%
- Rainfall amount: 20%
- Road accessibility: 10%

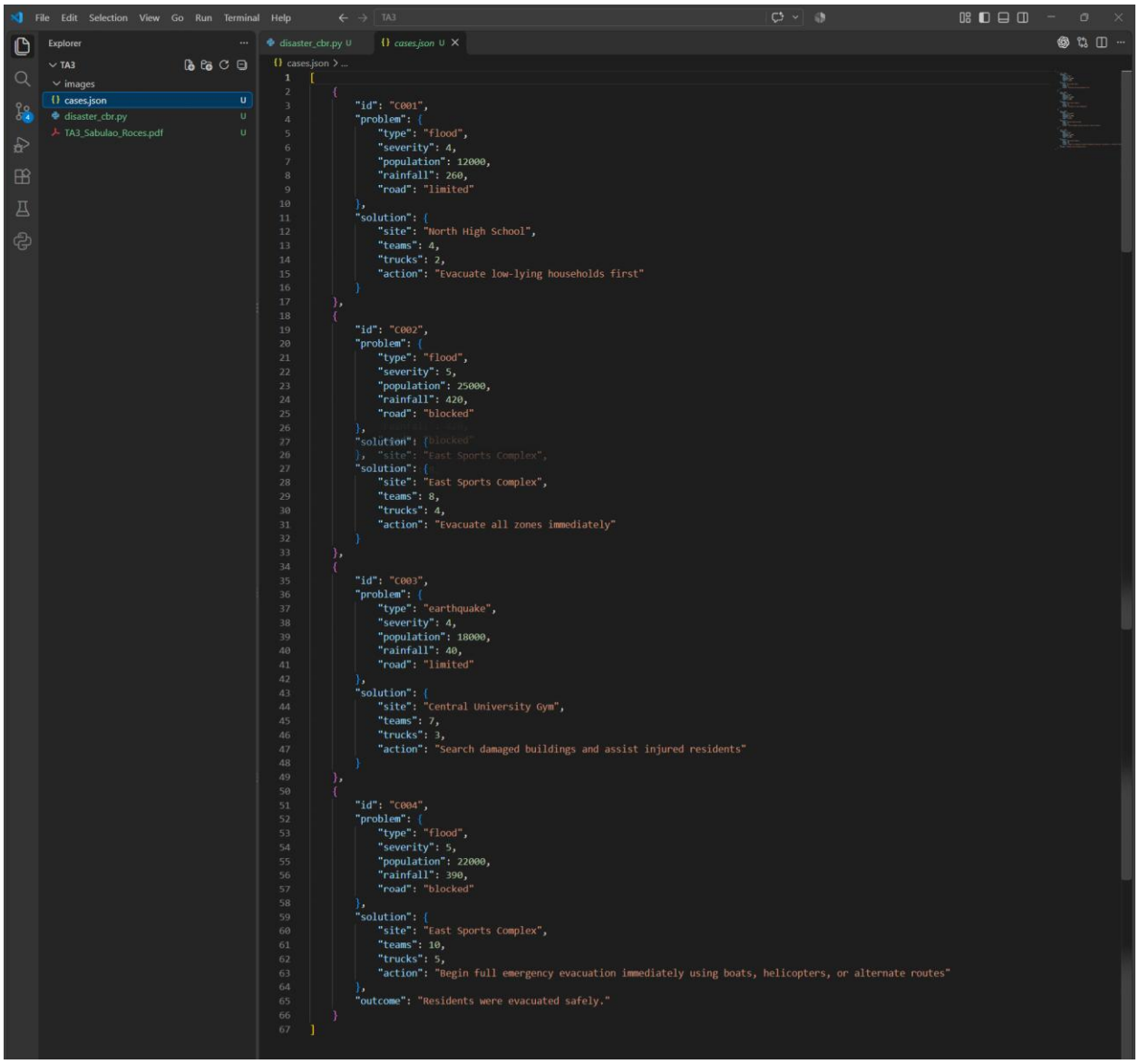
For categorical features, such as disaster type and road access, the system gives a score of 1 when the values match and 0 when they do not match. For numerical features, such as severity, population, and rainfall, the system calculates how close the new value is to the stored value. Higher similarity scores are given to values that are closer together. The scores are multiplied by their weights and added together. The case with the highest total score is retrieved. In the test scenario, Case C002 is retrieved because it is a severe flood with a large affected population and blocked roads.

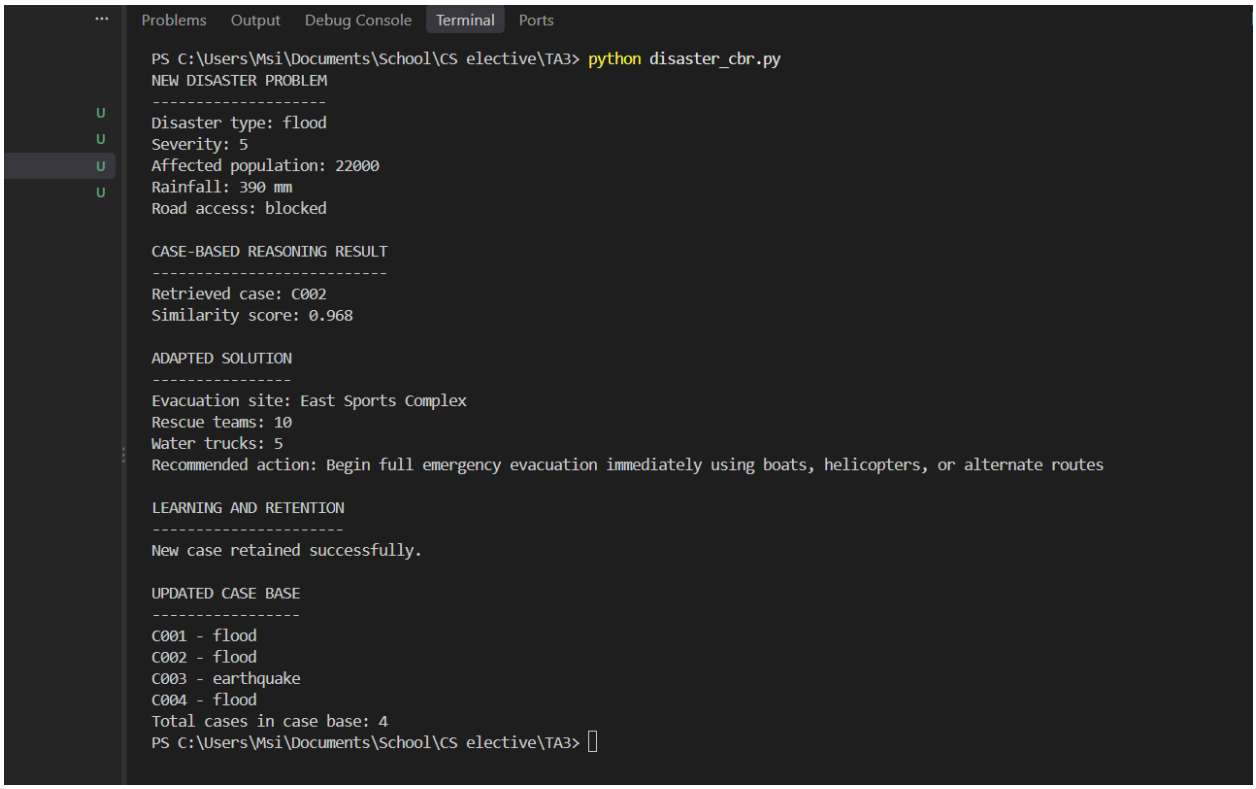
Adaptation Logic

After retrieving Case C002, the program adapts the previous solution to fit the new disaster conditions. If the affected population is greater than 20,000, the system adds two rescue teams and one water truck. If the severity level is 5, the system recommends full emergency evacuation. If the roads are blocked, the system recommends using boats, helicopters, or alternate routes. For the test scenario, the response plan changes from 8 rescue teams and 4 water trucks to 10 rescue teams and 5 water trucks. The recommended action is full emergency evacuation using alternate transportation.

Learning and Retention

After producing the adapted solution, the new disaster case is stored in the case base as Case C004. The program saves the updated case base in cases.json. This demonstrates the retention stage of Case-Based Reasoning because the system learns from the new experience and can use it in future situations.


File Edit Selection View Go Run Terminal Help
TA3
Explorer
TA3
images
cases.json U
disaster_cbr.py U
TA3_Sabulao_Roces.pdf U
disaster_cbr.py U
cases.json U
cases.json > ...
[
{
"id": "C001",
"problem": {
"type": "flood",
"severity": 4,
"population": 12000,
"rainfall": 260,
"road": "limited"
},
"solution": {
"site": "North High School",
"teams": 4,
"trucks": 2,
"action": "Evacuate low-lying households first"
}
},
{
"id": "C002",
"problem": {
"type": "flood",
"severity": 5,
"population": 25000,
"rainfall": 420,
"road": "blocked"
},
"solution": {
"site": "East Sports Complex",
"teams": 8,
"trucks": 4,
"action": "Evacuate all zones immediately"
}
},
{
"id": "C003",
"problem": {
"type": "earthquake",
"severity": 4,
"population": 18000,
"rainfall": 40,
"road": "limited"
},
"solution": {
"site": "Central University Gym",
"teams": 7,
"trucks": 3,
"action": "Search damaged buildings and assist injured residents"
}
},
{
"id": "C004",
"problem": {
"type": "flood",
"severity": 5,
"population": 22000,
"rainfall": 390,
"road": "blocked"
},
"solution": {
"site": "East Sports Complex",
"teams": 10,
"trucks": 5,
"action": "Begin full emergency evacuation immediately using boats, helicopters, or alternate routes"
},
"outcome": "Residents were evacuated safely."
}
]


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Msi\Documents\School\CS elective\TA3> python disaster_cbr.py
NEW DISASTER PROBLEM
Disaster type: flood
Severity: 5
Affected population: 22000
Rainfall: 390 mm
Road access: blocked
CASE-BASED REASONING RESULT
Retrieved case: C002
Similarity score: 0.968
ADAPTED SOLUTION
Evacuation site: East Sports Complex
Rescue teams: 10
Water trucks: 5
Recommended action: Begin full emergency evacuation immediately using boats, helicopters, or alternate routes
LEARNING AND RETENTION
New case retained successfully.
UPDATED CASE BASE
C001 - flood
C002 - flood
C003 - earthquake
C004 - flood
Total cases in case base: 4
PS C:\Users\Msi\Documents\School\CS elective\TA3>

C. Pair Reflection

My partner contributed by helping design the disaster-case attributes, preparing the sample cases, and checking the program output. I contributed by programming the similarity calculation, case retrieval, adaptation rules, and retention process. One challenge we encountered was deciding which features were most important when comparing disaster situations. We also had to determine how to compare numerical values such as population, rainfall, and severity. Through this activity, we learned that Case-Based Reasoning solves new problems by using knowledge from previous experiences. We learned that a retrieved solution must sometimes be adapted because each new situation has different conditions. Overall, the activity helped us understand how case representation, retrieval, adaptation, and learning work together.